

realtime-platform 二次开发指南（开发者手册）

适用工程：D:\realtime-platform\realtime-platform-main

版本：v1.0 编写日期：2026-09-12

前置阅读：《平台使用操作手册.md》《PLC 接入操作手册.md》《UPGRADE.md》

技术栈：Vue3 + Vite + Element Plus + ECharts + DataV（前端）；Express + Socket.IO + Prisma/MySQL + modbus-serial（后端）；Modbus TCP（PLC）

0 阅读前提与工程约定

约定	说明
单位与缩放	压力统一 $\times 10$ （0.1 kPa 精度）；温度 $\times 10$ 有符号；湿度 $\times 10$ 。 新增通道沿用此约定 ，避免 uint16 溢出
寄存器	协议原地址，不做 40001 偏移；H5U Easy 型只能用 D0~D8191（程序用 2001 段）
写入白名单	平台只写 30021（气囊压力设定）、30022（弓头高度设定）
定时器分工	采集定时器（真实/模拟）、控制定时器（双环 PID）相互独立，周期由 .env 配置
新增即隔离	优先新增文件 + 少量挂载行，便于回退（可删除挂载行恢复原状）
阈值可配	任何阈值/参数不得硬编码，统一放 .env 或 standards.js 配置区

1 开发环境与调试

1.1 推荐启动顺序

```
# 终端 1: PLC 模拟器（协议层模拟 H5U，带一阶滞后执行机构）
npm run plc-sim:closedloop      # 等价于 node plc-simulator.js --debug

# 终端 2: 后端 + 前端
npm run dev                    # 前端热更新（Vite）；后端另开 npm start
# 或生产形态
npm run build && npm start      # dist 由 Express 托管，单端口 3000
```

.env 关键项（联调用）：

```
PLC_MODE=real
PLC_HOST=127.0.0.1
PLC_PORT=1502
PLC_UNIT_ID=1
PLC_INTERVAL=1000
ZN_DEBUG=1
```

1.2 关键文件职责

文件	职责	改动风险
server.js	Express 服务、REST 接口、Socket.IO、采集/控制定时器、异常判定与告警入库	高（在运行核心）
plc-packet.js	寄存器编解码、主包/子包解析、阈值与状态字定义	中（纯函数，有测试价值）
plc-simulator.js	Modbus TCP 从站模拟器（寄存器/状态字/序号/子包/写回/一阶滞后）	低
control-engine.js	双环 PID v1：位置式并联、微分先行、条件抗饱和、Z-N 阶跃整定	中
control-engine-v2.js	v2：不完全微分、积分分离、斜坡、无扰切换、速度自适应、tuneSafe、PRESETS	低（新增）
standards.js	标准公式（目标接触力、限值）、告警分级配置、KPI 公式	低（新增）
alarm-engine.js	告警：滤波、变化率、连续确认、抑制、分级、确认/解除	低（新增）
link-monitor.js	丢包率、质量码、心跳、退避重连、写回读校验	低（新增）
maintenance.js	工单状态机、审计、健康指数、MTTR	低（新增）
upgrade-routes.js	/api/v2/* 路由聚合	低（新增）
src/**	前端页面、组件、路由、Socket 客户端	中
prisma/schema.prisma	数据模型（MySQL）	中（需迁移）

1.3 调试技巧

目的	方法
看原始寄存器	npm run plc-sim:closedloop（模拟器打印）+ GET /api/plc/packet
看整定过程	.env 设 ZN_DEBUG=1，平台控制台打印 y0/yInf/dY/tailPoints 等
看告警与链路	GET /api/v2/alarms、GET /api/v2/link、GET /api/v2/kpi
看数据库	npx prisma studio
前端实时数据	浏览器控制台 window.socket（src/socket.js 已挂到全局）观察推送
单元/基准测试	node tests/run-tests.js（36 项用例，输出 tests/report.json）

2 扩展一：新增传感器通道

以“新增 弓网接触力直接测量通道（或振动、位移）”为例，共 7 步。

2.1 PLC 侧：寄存器规划

1. 分配连续寄存器段（建议接在现有段之后，避免冲突）：

用途	建议地址（标准型）	Easy 型	缩放	方向
新通道均值	30006（30005 之后）	D2006	×10	读 FC03
新通道状态位	复用 30004 的 bit3（bit0~2 已占用）	同	—	读
新通道异常值	追加到子包 30016	D2016	×10	读

2. 修改 plc-program/H5U_main.st：增加 AD 通道→均值→组包逻辑，并在状态字 bit3 置位异常标志。

3. 同步 .env：

```
PLC_REG_NEW=30006
PLC_REG_SUB_BASE=30011      # 子包仍从此起，新增值依次排在其后
```

2.2 协议层：plc-packet.js 扩展

```
// 1) 阈值表增加新通道（与平台 analyzeFault 共用，避免两处漂移）
const THRESHOLDS = {
  tempHigh: 80, tempLow: -10,
  pressHigh: 1000, pressLow: 100,
  humidHigh: 85,
  newHigh: 300, newLow: 20,           // 新增：如接触力 N 或振动 g
};

// 2) 状态字位定义增加 bit3
const STATUS_BITS = {
  tempAnomaly: 1 << 0, pressAnomaly: 1 << 1, humidAnomaly: 1 << 2,
  newAnomaly: 1 << 3,                // 新增
  extended: 1 << 6, subValid: 1 << 7,
};

// 3) 定点编解码（沿用 ×10 约定）
function decodeNew(raw) { return toSigned16(raw) / 10; }
function encodeNew(val) { const s = Math.round(val * 10); return s < 0 ? s + 65536 : Math.min(65535, s); }

// 4) parseMainPacket 增加字段
function parseMainPacket(regs, extra) {
  // ...原有解析...
  const out = { /* temperature, pressure, humidity, status, seq, ... */ };
  if (extra && extra.newRaw !== undefined) out.newValue = decodeNew(extra.newRaw);
  return out;
}

// 5) 异常位图增加规则
function anomaliesFromBitmap(bitmap, values) {
  const anomalies = []; // ...原有三条...
  if (bitmap & STATUS_BITS.newAnomaly) {
    const v = values.newValue;
    const high = v > THRESHOLDS.newHigh;
```

```

        anomalies.push({
            type: high ? '新通道高限告警' : '新通道低限告警',
            level: 'error', value: v,
            detail: `[PLC 判定] ${v} ${high ? '>' : '<'} ${high ? THRESHOLDS.newHigh : THRESHOLDS.newLow}`,
        });
    }
    return anomalies;
}

module.exports = { /* 原导出 */ decodeNew, encodeNew };

```

2.3 服务端：server.js 采集与推送

```

// ① 采集侧：把新寄存器加入读取数组（沿用既有读取函数）
const regs = await client.readHoldingRegisters(PLC_CONFIG.regNew, 1);
const newValue = pkt.decodeNew(regs.data[0]);

// ② 判定侧：在 analyzeFault(data) 内增加判据（保持返回结构 {type, level, value, detail}）
if (data.newValue !== undefined) {
    if (data.newValue > pkt.THRESHOLDS.newHigh) faults.push({ type: '新通道高限告警',
        level: 'error', value: data.newValue, detail: '超上限' });
}

// ③ 推送侧：Socket.IO 广播时带上新字段
io.emit('sensorData', { t: Date.now(), temperature, pressure, humidity, newValue });

```

2.4 告警配置：standards.js + alarm-engine.js

```

// standards.js: 新增信号的分级与变化率阈值
const ALARM_DEFAULTS = {
    // ...temperature / pressure / humidity...
    newSignal: { warningHigh: 280, majorHigh: 320, criticalHigh: 360,
        warningLow: 30, majorLow: 20,
        rocPerMin: { warning: 20, major: 50 }, unit: 'N' },
};

// alarm-engine.js: 把新信号加入遍历列表（两处）
// _filter(): for (const sig of ['temperature','pressure','humidity','newSignal'])
// push(): for (const sig of ['temperature','pressure','humidity','newSignal'])

```

2.5 前端接入

1. src/socket.js 已建立同源连接，页面处订阅新字段：socket.on('sensorData', d => { this.newValue = d.newValue });
2. 在单车监测页新增一条曲线（ECharts series）或仪表盘组件（参考 src/components/DashboardGauge.vue）；
3. 若需独立页面：在 src/views/ 新建 .vue，并在 src/router/index.js 注册路由、src/components/LeftMenu.vue 加菜单项。

2.6 数据库与迁移

```

// prisma/schema.prisma（如需长期保存新通道）
model SensorSample {

```

```
id          Int          @id @default(autoincrement())
deviceId    String
t           DateTime
temperature Float?
pressure    Float?
humidity     Float?
newValue    Float?      // 新增通道
createdAt   DateTime @default(now()) @map("created_at")
@@map("sensor_sample")
}

npx prisma migrate dev --name add_sensor_sample
npx prisma generate
```

2.7 测试（必须补）

在 tests/ 下新增用例或在 tests/run-tests.js 增加断言：编解码往返误差 ≤0.05、越限钳位、位图→告警映射正确。

3 扩展二：新增自定义告警规则

3.1 支持的五类规则与实现位置

规则类型	说明	实现位置
阈值规则	高/低限分级	standards.ALARM_DEFAULTS + alarm-engine._grade
变化率规则	缓变早发现（本次升级新增）	alarm-engine._roc + _grade
组合规则	多信号与/或（如“压力低 且 温度高”）	alarm-engine.push 内新增判定
持续时间规则	越限持续 N 秒才告警	confirmN / 新增计时器
突变规则	与中值差超过阈值（抗噪）	_filter 输出与原始值差分

3.2 示例 A：接触力波动越限（组合/统计类）

```
// alarm-engine.js 构造函数中增加滑动窗口
this.win = { force: [] };

// push() 内：先算窗口统计，再判定
const w = this.win.force; w.push(sample.contactForce);
if (w.length > 30) w.shift();
if (w.length >= 10) {
  const mean = w.reduce((a, b) => a + b, 0) / w.length;
  const sd = Math.sqrt(w.reduce((a, b) => a + (b - mean) ** 2, 0) / w.length);
  const target = std.targetMeanContactForce(this.cfg.speed || 0).value;
  if (sd > 0.3 * target) { // 与 EN 50367 的 σ≤0.3Fm 判据一致
    this._emit({ signal: 'contactForce', level: 'major', value: +sd.toFixed(2),
                  roc: 0, source: 'statistics', message: `接触力波动
σ=${sd.toFixed(2)} N 超过 0.3Fm` });
  }
}
```

_emit() 建议抽取为统一出口：负责连续确认、抑制去重、入库与推送，避免多处重复逻辑。

3.3 示例 B：离线率越限（持续时间规则）

```
// 规则：1 分钟内离线次数 ≥3 次 → warning; ≥6 次 → major
this.offline = { win: [], };
pushOffline(t, isOffline) {
  this.offline.win.push({ t, isOffline });
  this.offline.win = this.offline.win.filter(x => t - x.t <= 60000);
  const n = this.offline.win.filter(x => x.isOffline).length;
  if (n >= 6) return this._emit({ signal: 'offline', level: 'major', value: n,
source: 'duration' });
  if (n >= 3) return this._emit({ signal: 'offline', level: 'warning', value: n,
source: 'duration' });
  return [];
}
```

3.4 参数调优建议

参数	作用	调优方向
confirmN	连续确认点数	误报多则增大（2→3），但会增加时延（约 1 s/点）
suppressMs	同源同级抑制窗口	告警刷屏则增大（2 min→5 min），但可能掩盖复发
rocWindowMs	变化率统计窗口	窗口越短越灵敏、越易受噪声影响；建议 30~60 s
medianOnly	仅中值滤波（低时延）	数据抖动大改 false（叠加均值），代价是时延增加

3.5 回归测试

```
// tests/run-tests.js 追加：注入缓变与阶跃，断言时延与级别
const eng = new AlarmEngine({ confirmN: 2 });
// 断言：缓慢漂移应在越限前给出 warning（ROC 判据）
```

4 扩展三：接入摄像头（视频 + 照片）

4.1 总体架构

现场相机(RTSP 554) →frp 隧道→ 云端流媒体(ZLMediaKit) →HTTP-FLV/WebRTC→ 浏览器页面
现场抓拍照片 →HTTP POST→ Express(/api/upload, multer) → uploads/ → 数据库存 URL

4.2 后端：照片上传与告警关联

既有 POST /api/upload（multer，字段名 file）已可用；把返回 URL 写入告警记录：

```
// 告警入库时写入图片/视频地址（schema 已预留 imageUrl / videoUrl）
await prisma.warningHistory.create({
  data: { code, content, time: new Date(t), level, value,
    imageUrl, // 上传接口返回的相对路径
    videoUrl, // 可选：事件片段地址
    carNumber, line, station, direction, location, mileage, speed },
});
```

若根目录 schema 尚无这些字段：把 realtime-platform/prisma/schema.prisma 中的扩展字段合并到根 schema，

再 `npx prisma migrate dev --name add_camera_fields`。

4.3 流媒体部署要点（ZLMediaKit）

```
# 云端（Docker 示例）
docker run -d --name zlm -p 1935:1935 -p 8080:80 -p 554:554 -p 8000:8000/udp \
  -p 10000:10000/udp zlmediakit/zlmediakit:master
# 拉流（把现场相机经 frp 映射到本机 7551）
ffmpeg -rtsp_transport tcp -i rtsp://127.0.0.1:7551/stream -c copy -f flv
rtmp://127.0.0.1/live/panto1
```

- 前端播放：`http://<云端 IP>:8080/live/panto1.live.flv`（flv.js）或 WebRTC 低延迟播放；
- 带宽估算：1 路 1080P H.264 $\approx 2\sim 4$ Mbps；现场上行带宽 $\geq$ 全部码流之和 $\times 1.5$ ；
- 录像策略：连续录像留在现场 NVR/SD 卡，云端只接收事件触发抓拍与告警片段。

4.4 前端接入

1. 页面中已预留 `videoUrl` 槽位（如 `src/views/SmartDetection/MonitorGlobal.vue`）：把地址绑定到播放器组件；
2. 播放器：flv.js（HTTP-FLV）或 WebRTC；引入后在 `onMounted` 建立播放、`onUnmounted` 销毁，断流自动重连；
3. 照片：告警详情页展示 `imageUrl`（``），点击放大。

4.5 国标（GB28181）方案

相机数量多或现场无法放网关时：云端部署 `wvp-GB28181-pro` + `ZLMediaKit`，相机 SIP 注册到云端，天然穿 NAT，无需 frp。

4.6 安全要求

- 554/502 不得直接暴露公网；隧道仅对云端 IP 开放；
- 播放地址加鉴权（token/有效期），对外数据脱敏；
- 用户权限：`users.role` 增加 `viewer/operator/admin` 三级，控制类接口校验 `operator` 以上。

4.7 本地测试（无真实相机）

```
ffmpeg -re -stream_loop -1 -i sample.mp4 -c copy -f rtsp
rtsp://127.0.0.1:8554/live/test
```

5 扩展四：把 v2 能力接进现有服务

5.1 挂载 /api/v2

```
// server.js
const createUpgradeRouter = require('./upgrade-routes');
app.use('/api/v2', createUpgradeRouter({ autoCreateOrder: true }));
```

5.2 控制回路切换到 v2

```

const { createControlEngineV2, tuneSafe, PRESETS } = require('./control-engine-v2');
const eng = createControlEngineV2({ system: 'AC', speed: 0 });

// 采集/工况变化时更新速度 → 目标接触力按标准公式自动调整
eng.setSpeed(currentSpeed);

// 控制定时器（原 controlLoopInterval 内）
controlLoopInterval = setInterval(() => {
  eng.tick({ pressure: latestPressure }, PLC_MODE === 'simulation' ? 'simulation' :
'plc');
  const st = eng.getStatus();
  // 写回（沿用既有写寄存器与回读校验）
  writeVerify(PLC_REG_WRITE_FORCE, Math.round(st.loops.force.output * 10));
  writeVerify(PLC_REG_WRITE_HEIGHT, Math.round(st.loops.posture.output));
}, CONTROL_INTERVAL);

```

整定按钮改为：

```

const g = tuneSafe(K, T, L); // T>L → ZN 折减; T≤L → IMC
eng.setParams('force', { kp: g.kp, ki: g.ki, kd: g.kd });
// 同时应用预设: derivFilterN / integralSeparation / rampRate (PRESETS.contactForce)

```

5.3 告警链路改造（保留原写库逻辑）

```

// 原: analyzeFault(data) → faults → handleAlarm(faults, avg) → 写 warning_history
// 新: 在 handleAlarm 内并联调用告警引擎（分级/变化率/抑制）
const newAlarms = alarmEngine.push({ t: Date.now(), ...avg });
for (const a of newAlarms) {
  await prisma.warningHistory.create({ data: {
    code: a.signal, content: a.message, time: new Date(a.t),
    level: a.level, value: a.value, remark: a.source,
  }});
  io.emit('addWarning', a); // 复用既有前端事件
  if (a.level === 'major' || a.level === 'critical') maint.createFromAlarm(a,
deviceId);
}

```

5.4 工单与审计表（DDL 示例）

```

CREATE TABLE work_order (
  id VARCHAR(16) PRIMARY KEY,
  device_id VARCHAR(64), alarm_id INT, level VARCHAR(16),
  description VARCHAR(255), status VARCHAR(16),
  assignee VARCHAR(64), created_at BIGINT, started_at BIGINT,
  completed_at BIGINT, verified_at BIGINT, remark VARCHAR(255)
);
CREATE TABLE work_order_audit (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  order_id VARCHAR(16), at BIGINT, action VARCHAR(16),
  worker VARCHAR(64), note VARCHAR(255)
);

```

6 扩展五：新增接口与前端页面

6.1 后端接口模板

```

app.get('/api/device/:id/health', async (req, res) => {
  const id = req.params.id;
  if (!id) return res.status(400).json({ ok: false, error: '缺少设备编号' });
  try {
    const h = maint.healthIndex({ forceMean: Number(req.query.forceMean),
forceTarget: 100 });
    res.json({ ok: true, deviceId: id, ...h });
  } catch (e) {
    res.status(500).json({ ok: false, error: String(e.message) || e });
  }
});

```

约定：成功 { ok: true, ... }; 失败 { ok: false, error } + 合适状态码。

6.2 前端新增页面

```

// src/router/index.js
{ path: 'device_health', name: 'DeviceHealth', component: () =>
import('@/views/SmartDetection/DeviceHealth.vue') }

<!-- src/views/SmartDetection/DeviceHealth.vue (骨架) -->
<template>
  <div class="page">
    <el-table :data="rows" border>
      <el-table-column prop="deviceId" label="设备" />
      <el-table-column prop="score" label="健康指数" />
      <el-table-column prop="grade" label="等级" />
      <el-table-column prop="advice" label="建议" />
    </el-table>
  </div>
</template>
<script setup>
import { ref, onMounted } from 'vue'
import api from '@/utils/api'
const rows = ref([])
onMounted(async () => { rows.value = (await api.get('/api/v2/health')).data })
</script>

```

菜单项加到 src/components/LeftMenu.vue；接口统一走 src/utils/api.js（同源 baseUrl： '/', 避免硬编码 localhost）。

6.3 Socket.IO 事件约定

方向	事件	载荷
服务端→前端	sensorData	{ t, temperature, pressure, humidity, ... }
服务端→前端	addWarning	告警对象（含 id/level/signal/value/message）
服务端→前端	updatePie	统计分布数据
前端→服务端	addWarning（既有）	手工新增告警

新增事件请遵守：① 名称用 camelCase；② 载荷为纯 JSON；③ 断线重连由 socket.io-client 自动处理，业务侧只处理数据。

7 编码规范与提交要求

- 1. 命名：文件用 kebab 或既有风格；函数动词开头（parseMainPacket、tuneSafe）；常量全大写。
- 2. 注释与依据：涉及标准/公式处必须注明来源与版本（如“EN 50367 / TB/T 3271，系数需按采用版本复核”）。
- 3. 配置优先：阈值、周期、限幅不得硬编码。
- 4. 纯函数化：编解码、判定、指标计算尽量写成纯函数，便于测试。
- 5. 测试：新增/修改模块须补 tests/ 用例，提交前跑 node tests/run-tests.js 全绿。
- 6. 回退设计：优先“新增文件 + 一行挂载”，删除挂载行即回退；改动 server.js 前先备份（工程内已有 server.js.full.txt 参考）。
- 7. 安全：新增写操作必须走白名单 + 回读校验 + 审计；不得扩大寄存器写入范围。

8 常见开发问题（FAQ）

问题	原因	解决
模拟器连不上	平台指向了 502 而模拟器在 1502	统一 PLC_PORT；确认模拟器进程与防火墙
读到的值与 PLC 不一致	地址偏移 1 或误加 40001	用协议原地址；用 ModbusPoll 交叉验证
写入无效	写丢、寄存器只读、映射错	用 writeVerify 查看回读；核对 MBS 映射与写权限
序列号乱跳	模拟器/PLC 复位或回绕	序号按 65536 回绕属正常；持续乱跳查固件与链路
前端拿不到新字段	后端未推送或字段名不一致	服务端 io.emit 与前端订阅字段保持同名
图表卡顿	推送频率过高	降采样或前端节流（如 500 ms 聚合）
Prisma 报表不存在	未迁移	npx prisma migrate deploy
中文乱码	终端/文件编码	统一 UTF-8；Windows 终端用 chcp 65001

9 附录

9.1 目录速查

realtime-platform-main/	
├ server.js	后端服务（接口、Socket、定时器、告警入库）
├ plc-packet.js	寄存器编解码与异常位图（与模拟器共用）
├ plc-simulator.js	PLC 模拟器（Modbus 从站）
├ control-engine.js	双环 PID v1 + Z-N 整定
├ control-engine-v2.js	v2：安全整定/滤波/斜坡/速度自适应（新增）
├ standards.js	标准公式、告警分级、KPI（新增）
├ alarm-engine.js	告警引擎（新增）

└ link-monitor.js	链路质量与写回读校验（新增）
└ maintenance.js	工单/健康指数/MTTR（新增）
└ upgrade-routes.js	/api/v2 路由（新增）
└ tests/run-tests.js	测试与基准（新增）
└ plc-program/H5U_main.st	PLC 程序（AutoShop ST）
└ prisma/schema.prisma	数据模型（MySQL）
└ src/	前端（views/components/router/utils）
└ uploads/	照片与附件

9.2 标准与公式速查

项目	公式
目标平均接触力（AC/DC）	$F_m = 0.00097 \cdot v^2 + 70$ / $F_m = 0.00155 \cdot v^2 + 70$ (N, v km/h)
动态判据	$\sigma \leq 0.3F_m$, $F_{max} \leq F_m + 3\sigma$, $F_{min} \geq F_m - 3\sigma$
接触力—气压	$F = p \cdot A_{eff} - F_{offset}$
弓头高度	$H = L1 \cdot \sin\theta1 + L2 \cdot \sin(r \cdot \theta1)$
Z-N	$K_p = 1.2T / (K \cdot L)$, $T_i = 2L$, $T_d = 0.5L$
IMC-PI	$K_c = T / (K(\lambda + L))$, $T_i = T$
安全折减	$r = 1 / (1 + L/T)$ (限幅 0.4~1)
KPI	可用率 1 - 停机/总时长；完整率 有效/应有；丢包率 缺口/应到；MTTR 修复总时长/工单数

9.3 相关文档

文档	用途
平台使用操作手册.md	安装、配置、三种模式、日常操作、排错
一线检修速查卡.docx	现场一页速查（告警处置、常见故障）
PLC 接入操作手册.md	接线、AutoShop、寄存器、联调
云服务器部署方案.md	部署、隧道、视频、成本
UPGRADE.md	v2 升级需求对照与测试结果

修订记录

版本	日期	说明
v1.0	2026-09-12	首版：开发环境、五类扩展（传感器/告警规则/摄像头/v2 集成/接口页面）、规范与 FAQ